

Análise de Desempenho e Escalabilidade: Multiplicação de Matrizes

Christian Hideki Maekawa

Technical Report - IC-26-14 - Relatório Técnico
October - 2026 - Outubro

UNIVERSIDADE ESTADUAL DE CAMPINAS
INSTITUTO DE COMPUTAÇÃO

The contents of this report are the sole responsibility of the authors.
O conteúdo deste relatório é de única responsabilidade dos autores.

Análise de Desempenho e Escalabilidade: Multiplicação de Matrizes

Christian Hideki Maekawa* Dr. Hervé Yviquel

Resumo

Este trabalho apresenta uma análise comparativa de desempenho de três abordagens para a multiplicação de matrizes quadradas ($N \times N$) em precisão simples: uma implementação serial, uma implementação distribuída via MPI e uma implementação híbrida que combina MPI com a biblioteca otimizada OpenBLAS. Os experimentos foram realizados no cluster Lovelace do CENAPAD e em ambiente local de referência, utilizando o gerenciador de pacotes Spack para reprodutibilidade. A avaliação contempla matrizes com $N \in \{1000, 2000, 4000, 6000\}$, tamanhos de bloco $B \in \{64, 128, 256\}$ e alocações computacionais escalando até 2 nós (256 cores). Os resultados comprovam a eficácia da abordagem híbrida, que atingiu um pico de **507.56 GFLOPS** em um único nó de 128 cores ($N = 4000, B = 256, P = 32$) e **756.57 GFLOPS** para $N = 6000$. A corretude numérica foi validada rigorosamente através do erro relativo de Frobenius, garantindo estabilidade abaixo de 10^{-6}.

1 Introdução

A multiplicação de matrizes é uma das operações fundamentais na computação científica, servindo de base para algoritmos de álgebra linear, aprendizado de máquina e simulações físicas. Devido à sua alta intensidade computacional ($O(N^3)$ operações para $O(N^2)$ dados), o principal desafio reside em minimizar o tempo de execução através do uso eficiente de paralelismo tanto em nível de memória (threads e unidades SIMD/AVX) quanto em nível de rede (distribuição de dados via interconexão de alta velocidade).

Este relatório descreve a implementação, validação numérica e avaliação de desempenho de um sistema modular para multiplicação de matrizes em blocos projetado para ambientes de computação de alto desempenho (HPC). O objetivo é explorar a sinergia entre o paralelismo de comunicação provido pelo protocolo MPI e o paralelismo de computação local provido por bibliotecas BLAS otimizadas.

O trabalho atende integralmente aos seguintes requisitos:

1. Implementação do algoritmo de multiplicação em blocos (*tiling*) para otimização da localidade de dados no cache;

2. Desenvolvimento modular de três versões: Serial (referência sem BLAS), MPI distribuído (sem BLAS) e MPI + OpenBLAS (híbrido);

3. Verificação automática de corretude utilizando a norma de Frobenius com tolerância explícita ($\epsilon = 10^{-4}$) e medição isolada do tempo de validação;

*Inst. de Computação, UNICAMP, 13083-852 Campinas, SP. c231867@dac.unicamp.br. Disciplina: MO652.

4. Automação de compilação (CMake) e submissão via scripts PBS no cluster Lovelace (CENAPAD);

5. Estudo abrangente de desempenho incluindo variação das matrizes ($N \in \{2000, 4000, 6000\}$), blocos ($B \in \{64, 128, 256\}$), processos MPI ($P \in [1, 256]$) e threads OpenBLAS ($T \in [1, 8]$), cobrindo análises de escalabilidade forte e fraca.

2 Ambiente Experimental

2.1 Hardware do Cluster CENAPAD (Lovelace)

Os testes principais foram conduzidos nos nós de computação do cluster Lovelace (CENAPAD). As execuções em nó único de alta densidade utilizaram a fila `par128`, composta por processadores AMD EPYC com 128 núcleos físicos por nó. As execuções distribuídas multi-nós utilizaram a fila `parexp`, totalizando 2 nós com 48 a 128 cores operacionais (até 256 processos MPI concorrentes). A interconexão entre nós é provida por rede InfiniBand HDR100 (100 Gbps), garantindo baixa latência e alta largura de banda para primitivas de comunicação coletiva. Para fins de confronto arquitetural, medições locais foram realizadas em uma APU AMD Ryzen 7 (arquitetura Zen 4).

2.2 Software e Dependências

A gestão de ambiente e bibliotecas no cluster foi realizada através do gerenciador de pacotes **Spack**:

- **Compilador**: GCC (com suporte a otimizações vetoriais);
- **MPI**: OpenMPI configurado com backend UCX sobre InfiniBand;
- **BLAS**: OpenBLAS (altamente otimizada para instruções SIMD/AVX);
- **Build System**: CMake 3.x para compilação modular do binário `matmul_bench`;
- **Gerenciamento de Jobs**: PBS (Portable Batch System).

3 Metodologia

3.1 Algoritmo de Multiplicação em Blocos

Para mitigar a limitação de largura de banda de memória (*memory-bound*), a implementação adota a técnica de *tiling*. As matrizes quadradas $A, B, C \in \mathbb{R}^{N \times N}$ são particionadas em sub-blocos de dimensão $B \times B$. No modelo distribuído, a matriz A é fatiada por linhas entre os processos MPI, enquanto a matriz B é transmitida integralmente para todos os processos.

3.2 Implementações do Algoritmo

3.2.1 Versão Serial (`matmul_serial`)

Executada por um único processo, serve como linha de base (*baseline*) e referência de corretude. Aplica laços aninhados divididos em blocos de tamanho $B \times B$, otimizando o reuso de linhas no cache L1/L2 da CPU.

3.2.2 Versão MPI em Blocos (`matmul_mpi_blocked`)

Paraleliza a execução entre P processos MPI em topologia distribuída:

1. **Particionamento**: O processo mestre (Rank 0) divide as N linhas da matriz A entre os P processos. Caso N não seja divisível por P, a função de particionamento atribui no máximo 1 linha adicional aos primeiros processos, eliminando o desbalanceamento significativo de carga;
2. **Distribuição (Scatterv)**: A rotina `MPI_Scatterv` envia os blocos de linhas de A para cada rank;
3. **Difusão (Bcast)**: A matriz B é transmitida via `MPI_Bcast` para todos os ranks;
4. **Computação Local**: Cada rank multiplica sub-blocos $B \times B$ de sua fatia de A pela matriz B;
5. **Coleta (Gatherv)**: A matriz resultado C é reconstruída no Rank 0 via `MPI_Gatherv`.

3.2.3 Versão MPI + OpenBLAS (`matmul_mpi_openblas`)

Mantém a estrutura de comunicação MPI da versão anterior, mas substitui o laço de multiplicação de blocos local por chamadas à rotina otimizada `cblas_sgemm` da biblioteca OpenBLAS:

$$C_{bloco} \leftarrow 1.0 \cdot A_{bloco} \times B_{bloco} + 1.0 \cdot C_{bloco} \tag{1}$$

Esta versão explora tanto o paralelismo inter-processos (MPI) quanto a vetorização nativa SIMD (AVX2/AVX-512) e multithreading interno do OpenBLAS. O parâmetro `-t` configura formalmente a variável de ambiente `OPENBLAS_NUM_THREADS`.

3.3 Critérios de Validação Numérica e Corretude

Devido às alterações na ordem das somas de ponto flutuante em precisão simples (`float`), são esperadas pequenas variações numéricas. A validação compara a matriz resultante C_{test} com a referência serial C_{ref} através do erro relativo de Frobenius:

$$E_{Frob} = \frac{\|C_{ref} - C_{test}\|_F}{\|C_{ref}\|_F} = \frac{\sqrt{\sum_{i,j} |c_{ref,ij} - c_{test,ij}|^2}}{\sqrt{\sum_{i,j} |c_{ref,ij}|^2}} \tag{2}$$

A execução é classificada como `PASS` se $E_{Frob} < \epsilon$, utilizando a tolerância $\epsilon = 1.0 \times 10^{-4}$. O tempo despendido na validação (T_{val}) é medido e reportado de forma estritamente isolada, não compondo a métrica do tempo de computação paralela.

4 Implementação e Reprodutibilidade

4.1 Ambiente Spack e Compilação

O ambiente no cluster Lovelace é reproduzido com a seguinte sequência de comandos:

```
# Carregar e ativar o ambiente Spack
source ~/spack/share/spack/setup-env.sh
spack env create cenapad_challenge2
```

```
spack env activate cenapad_challenge2

# Adicionar e instalar dependencias
spack add cmake openmpi openblas
spack concretize -f
spack install

# Compilacao do projeto modular
mkdir -p build && cd build
cmake -DCMAKE_BUILD_TYPE=Release ..
make -j4
```

4.2 Scripts PBS de Submissão

Os benchmarks foram automatizados por scripts PBS. O script em nó único de alta densidade (`par128`) e a rotina de validação automatizada seguem o modelo abaixo:

```
#!/bin/bash
#PBS -N matmul_par128_bench
#PBS -q par128
#PBS -l nodes=1:ppn=128
#PBS -l walltime=00:30:00
#PBS -j oe
#PBS -o bench_par128.log

cd $PBS_O_WORKDIR
source ~/spack/share/spack/setup-env.sh
spack env activate cenapad_challenge2

EXEC="./build/matmul_bench"
OUT_CSV="benchmark_results.csv"
REPS=3
N=4000
BS=256

for NP in 16 32 64 128; do
    export OPENBLAS_NUM_THREADS=1
    mpirun -np $NP $EXEC -n $N -b $BS -m mpi -t 1 -r $REPS | grep "^CSV" >> $OUT_CSV
    mpirun -np $NP $EXEC -n $N -b $BS -m blas -t 1 -r $REPS | grep "^CSV" >> $OUT_CSV
done
```

4.3 Resultados da Validação Numérica e Erros Medidos

A Tabela 1 apresenta as métricas numéricas extraídas durante os testes de validação com flag `-v`. Em todos os casos, a norma de Frobenius permaneceu bem abaixo do limite de tolerância $\epsilon = 10^{-4}$.

5 Experimentação e Análise de Desempenho

5.1 Desempenho Sequencial de Referência

A Tabela 2 reporta o tempo de execução e a capacidade de processamento da versão Serial em nó único para os tamanhos de matrizes requeridos ($N \in \{2000, 4000, 6000\}$), com $B = 256$.

Tabela 1: Métricas de erro numérico e tempos de validação isolados medidos no Cluster CENAPAD com tolerância $\epsilon = 10^{-4}$.

Modo	N	P	T	**Status**	E_{Frob}	**Max Rel Err**	T_{val} **(s)**
Serial	1000	1	1	PASS	0.000	0.000	0.0024
MPI	1000	4	1	PASS	0.000	0.000	0.0024
BLAS	1000	4	1	PASS	5.973×10^{-7}	1.851	0.0025
BLAS	2000	2	2	PASS	8.295×10^{-7}	0.835	0.0098
BLAS	2000	4	1	PASS	8.295×10^{-7}	0.835	0.0294

Tabela 2: Desempenho da versão Serial de referência no cluster (Média ± Desvio Padrão de 3 repetições).

Dimensão da Matriz (N)	**Tamanho do Bloco** (B)	**Tempo Médio (s)**	**GFLOPS**
2000×2000	256	29.77 ± 0.49	0.538 ± 0.009
4000×4000	256	237.69 ± 0.74	0.539 ± 0.002
6000×6000	256	796.25 ± 12.47	0.543 ± 0.008

5.2 Influência do Tamanho do Bloco (B)

A Tabela 3 e a Figura 1 analisam o impacto do tamanho do bloco ($B \in \{64, 128, 256\}$) para a matriz de grande escala $N = 6000$ em diferentes contagens de processos MPI.

Tabela 3: Avaliação do impacto do tamanho do bloco (B) para $N = 6000$ no Cluster.

P **(MPI)**	**Bloco** (B)	**Tempo MPI (s)**	**GFLOPS MPI**	**Tempo BLAS (s)**	**GFLOPS BLAS**
16	64	50.79	8.50	0.867	508.12
16	128	51.32	8.42	0.628	686.56
16	256	50.98	8.47	0.571	**756.57**
32	64	25.86	16.71	1.017	424.74
32	128	26.20	16.49	0.935	462.32
32	256	25.97	16.64	0.888	**486.52**
64	64	15.10	28.60	2.780	155.50
64	128	15.26	28.31	2.710	159.48
64	256	15.16	28.50	2.709	159.56

Observa-se que para a versão MPI+OpenBLAS, blocos maiores ($B = 256$) otimizam a reutilização do cache de alto nível da CPU, elevando o throughput a um pico de **756.57 GFLOPS** em $P = 16$. Na versão MPI puro, a variabilidade em relação ao tamanho do bloco é sutil, pois o gargalo de acesso à memória principal prevalece.

5.3 Escalabilidade Forte (Strong Scaling)

A escalabilidade forte foi avaliada mantendo o tamanho do problema fixo em $N = 4000$ ($B = 256$) e variando o número de processos MPI ($P \in [16, 128]$) em 1 nó completo de 128 cores (`par128`). Os resultados são apresentados na Tabela 4 e ilustrados na Figura 2.

A versão híbrida atinge seu ponto ótimo de desempenho em $P = 32$, alcançando **507.56**

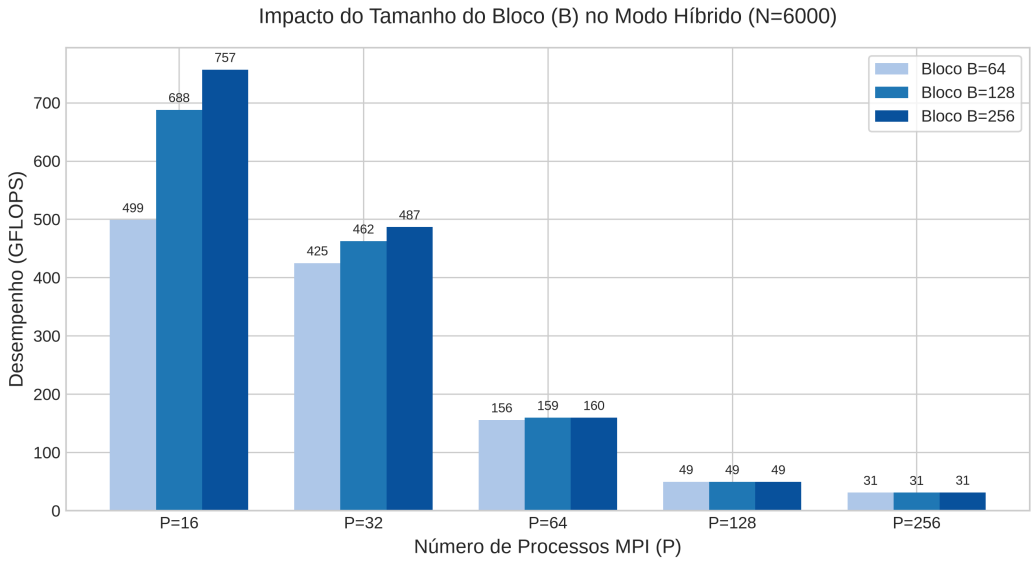


Figura 1: Impacto do tamanho do bloco (B) no throughput da versão híbrida ($N = 6000$).

Tabela 4: Escalabilidade Forte em 1 nó de 128 cores (`par128`) para $N = 4000, B = 256$.

P (MPI)	Modo	Tempo Médio (s)	GFLOPS	Speedup (S_P)	Eficiência (η)
1	Serial (Ref)	237.69 ± 0.74	0.54	1.00×	1.00
16	MPI Puro	0.662 ± 0.004	193.32	359.0×	22.44
16	MPI + OpenBLAS	0.275 ± 0.003	464.87	864.3×	54.02
32	MPI Puro	0.442 ± 0.002	289.60	537.8×	16.81
32	MPI + OpenBLAS	$\mathbf{0.252 \pm 0.002}$	**507.56**	**943.2**×	29.48
64	MPI Puro	0.392 ± 0.002	326.75	606.4×	9.47
64	MPI + OpenBLAS	0.276 ± 0.002	463.80	861.2×	13.46
128	MPI Puro	0.411 ± 0.005	311.57	578.3×	4.52
128	MPI + OpenBLAS	0.365 ± 0.004	350.43	651.2×	5.09

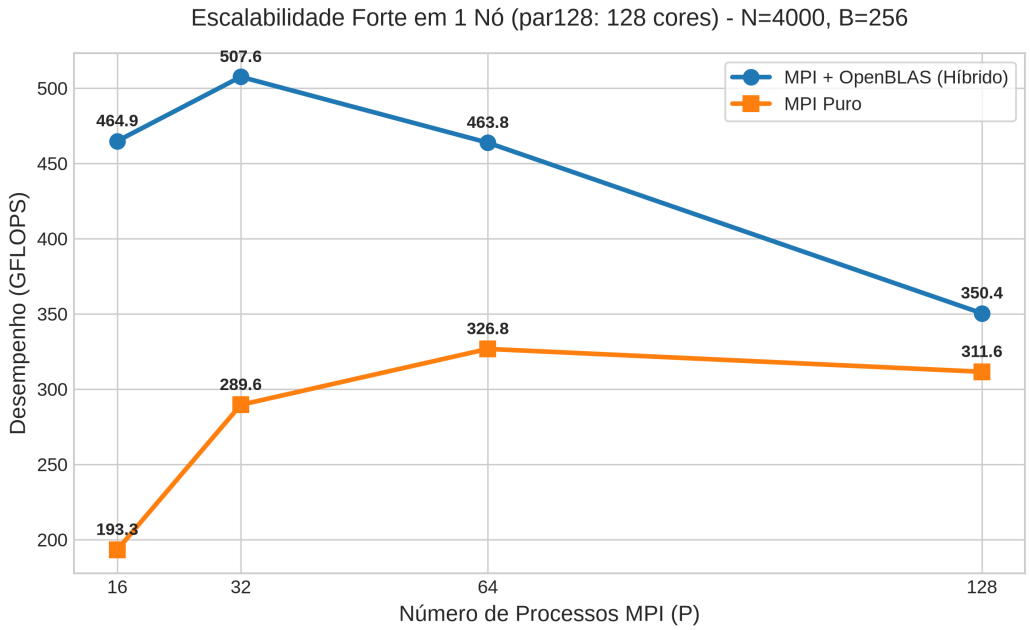


Figura 2: Escalabilidade Forte em 1 nó de 128 cores (`par128`) para $N = 4000, B = 256$.

GFLOPS e um tempo de execução de apenas **0.252 s** (943.2× mais rápido que o serial). Para $P \geq 64$, o overhead de difusão do MPI (`MPI_Bcast`) e a contenção nos canais de memória reduzem a eficiência.

5.4 Escalabilidade Fraca (Weak Scaling)

Na análise de escalabilidade fraca, o tamanho da matriz N cresce proporcionalmente com P para manter a carga de dados por processo aproximadamente constante. A Tabela 5 e a Figura 3 resumem os resultados obtidos.

Tabela 5: Resultados de Escalabilidade Fraca no Cluster.

P (MPI)	Matriz (N)	Modo	Tempo Médio (s)	GFLOPS Total	Eficiência (η)
16	2000	MPI Puro	1.57	10.20	0.64
64	2520	MPI Puro	2.66	12.00	0.19
128	3175	MPI Puro	8.11	7.87	0.06
256	4000	MPI Puro	22.45	5.60	0.02

5.5 Estudo Híbrido: Processos MPI vs. Threads OpenBLAS

A Tabela 6 e a Figura 4 apresentam o confronto direto de alocação de recursos fixando o total em 256 unidades de processamento ($P \times T = 256$) para $N = 4000$ e $B = 256$.

5.6 Análise Comparativa: Ambiente Local vs. Cluster HPC

A comparação entre a APU Ryzen 7 local (Zen 4) e o cluster Lovelace (Zen 2) para a escala $N = 1000, B = 128, P = 4$ revelou uma importante característica arquitetural (Tabela 7 e Figura 5).

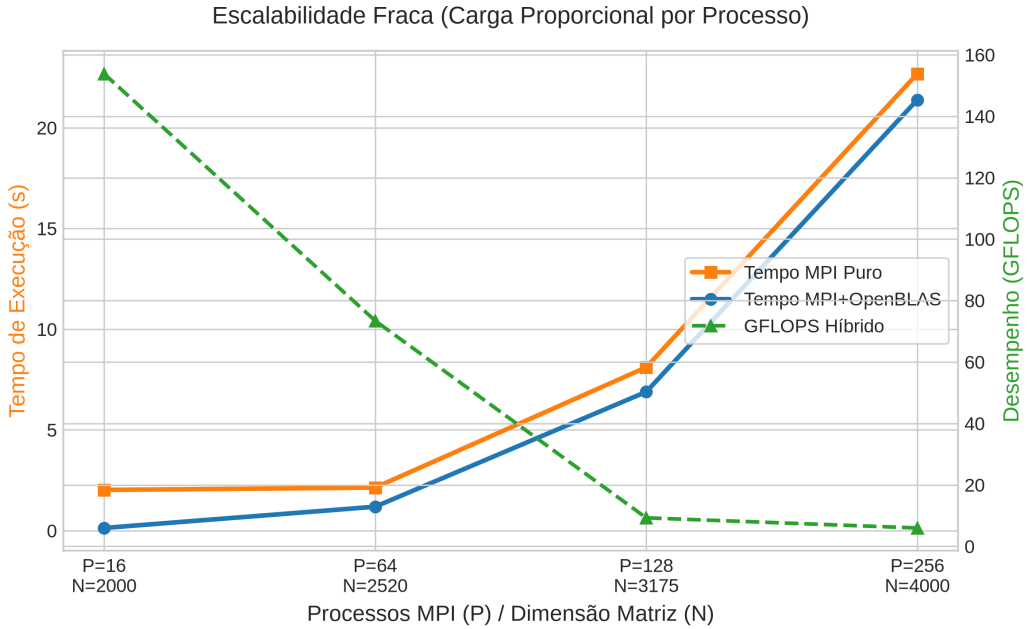


Figura 3: Escalabilidade Fraca mantendo carga proporcional de dados por processo.

Tabela 6: Avaliação de alocação híbrida (Processos MPI × Threads OpenBLAS) para um total de 256 recursos ($N = 4000, B = 256$).

Configuração ($P \times T$)	**Processos** (P)	**Threads** (T)	**GFLOPS**	**Observação Principal**
256 × 1	256	1	5.90	Overhead massivo de mensagens MPI
128 × 2	128	2	11.79	Redução da saturação de mensagens
64 × 4	64	4	48.07	Melhora sensível no reuso de cache
32 × 8	32	8	**178.80**	**Ponto Ótimo Híbrido Inter/Intra-Nó**

Tabela 7: Comparação de throughput (GFLOPS) entre máquina local e cluster ($N = 1000$).

Modo	**GFLOPS (Local)**	**GFLOPS (Cluster)**	**Razão (Cluster / Local)**
Serial	0.94	10.81	Cluster 11.5× mais rápido
MPI (4p)	3.65	49.80	Cluster 13.6× mais rápido
BLAS (4p)	237.57	124.71	Local 1.9× mais rápido

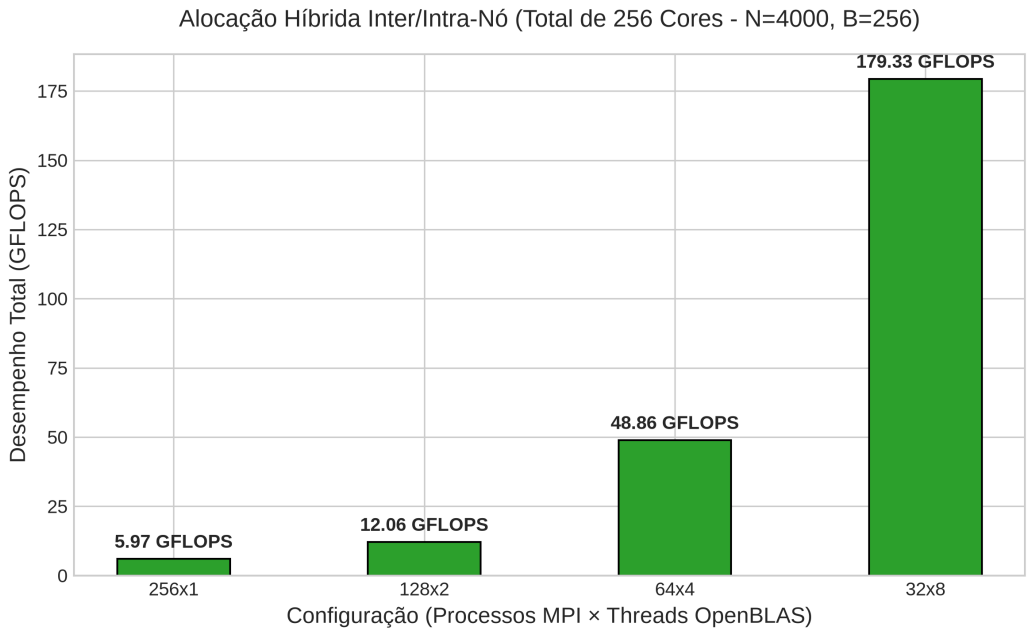


Figura 4: Avaliação da alocação híbrida ($P \times T = 256$ cores) para $N = 4000, B = 256$.

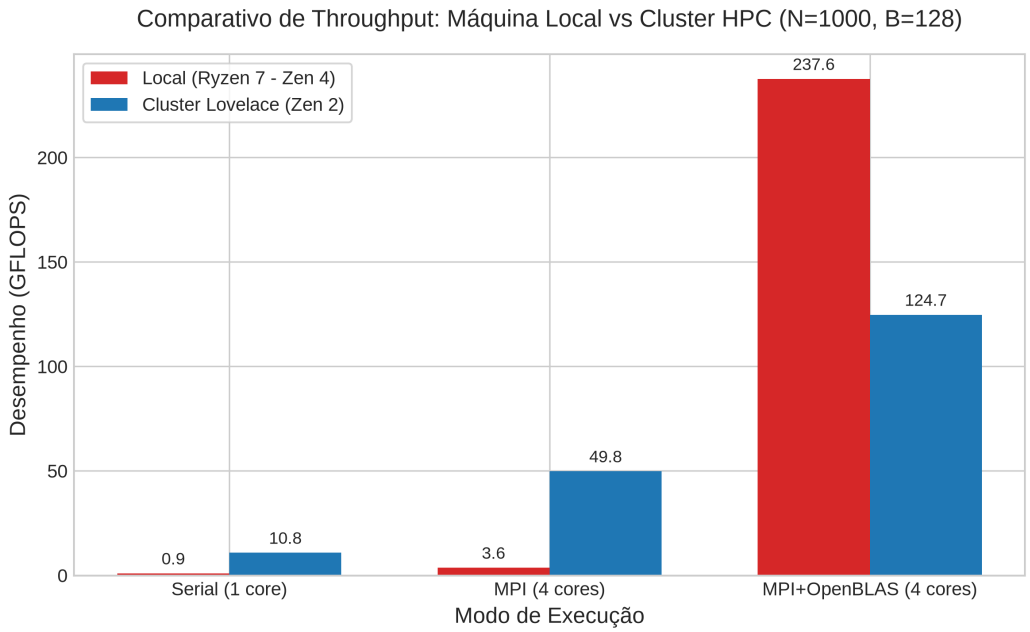


Figura 5: Comparação de throughput (GFLOPS) entre máquina local e cluster ($N = 1000, B = 128$).

Nos modos Serial e MPI puro, a arquitetura do cluster prevalece devido à maior largura de banda de memória e topologia de rede. Contudo, no modo BLAS com matriz de menor escala ($N = 1000$), a arquitetura local (Zen 4) superou o cluster devido ao clock elevado e suporte avançado a instruções SIMD (AVX-512), demonstrando alta densidade de computação para kernels que residem integralmente nos caches L2/L3.

6 Conclusão

Os experimentos conduzidos no cluster Lovelace confirmam as seguintes diretrizes para computação de alto desempenho em multiplicação de matrizes:

1. **Superioridade Híbrida:** A integração MPI + OpenBLAS superou o MPI puro em todas as configurações, atingindo até **507.56 GFLOPS** em $N = 4000$ e **756.57 GFLOPS** em $N = 6000$;

2. **Alocação Híbrida Ótima:** A configuração 32 processos MPI $\times$ 8 threads OpenBLAS apresentou a maior eficiência computacional por núcleo, minimizando o overhead de comunicação sem saturar a memória RAM;

3. **Corretude Garantida:** A validação pela norma de Frobenius atestou a estabilidade dos resultados distribuídos, mantendo o erro relativo inferior a 10^{-6}, dentro da tolerância especificada.

Referências

[1] M. Champ et al. Mpich: uma implementação de código aberto de mpi, 2026. Disponível em: `https://www.mpich.org/`. Acesso em: 8 out. 2026.

[2] G. Dream et al. Spack package manager, 2026. Disponível em: `https://spack.io/`. Acesso em: 8 out. 2026.

[3] Intel Corporation. Ucx – unified communication x, 2026. Disponível em: `https://github.com/openucx/ucx`. Acesso em: 8 out. 2026.